Module 13 – Containerization using Docker

Docker Overview

What is Docker?

Docker is an open-source containerization platform used to develop, package, and run applications inside lightweight containers. Containers include the application along with all its dependencies, ensuring it runs consistently across different environments.

Features of Docker

- Lightweight and fast.
- Portable across different platforms.
- Easy application deployment.
- Efficient resource utilization.
- Application isolation.
- Supports microservices architecture.
- Easy scalability and automation.

Docker Commands

Docker provides commands to manage images and containers.

Command	**Description**
docker run	Creates and starts a container.
docker ps	Lists running containers.
docker stop	Stops a running container.
docker rm	Removes a container.
docker images	Lists available images.
docker rmi	Removes an image.
docker pull	Downloads an image from Docker Hub.
docker exec	Executes a command inside a running container.
docker run <image> <command>	Runs a container and executes a specific command.

Docker Run

The docker run command creates and starts a new container from an image.

Common Usage

- **Run a container:** docker run nginx
- **Run with a specific name:** docker run --name mycontainer nginx
- **Run in detached mode:** docker run -d nginx
- **Run interactively:** docker run -it ubuntu bash
- **Publish container ports:** docker run -p 8080:80 nginx
- **Remove container after exit:** docker run --rm ubuntu

Docker Images

What is a Docker Image?

A Docker image is a read-only template that contains the application, libraries, dependencies, and configuration required to create a container.

Key Concepts

- **Image Layers:** Images are built in multiple reusable layers.
- **Container Layer:** A writable layer added when a container starts.
- **Base Image:** The initial image (e.g., Ubuntu, Alpine).
- **Parent Image:** The image from which another image is created.
- **Docker Manifest:** Stores metadata about Docker images.
- **Container Registry:** A service for storing Docker images (e.g., Docker Hub).
- **Container Repository:** A collection of Docker images.

Creating a Docker Image

1. **Interactive Method:** Create a container, make changes, and commit them as an image.
2. **Dockerfile Method:** Use a Dockerfile containing build instructions (recommended).

Docker Build Context

The Docker build context is the set of files and folders sent to Docker during image creation.

Docker Compose

What is Docker Compose?

Docker Compose is a tool used to define and manage multi-container Docker applications using a single YAML configuration file.

Benefits

- Manage multiple containers together.

- Simplifies deployment.
- Easy configuration.
- Supports automated startup and shutdown.

Basic Docker Compose Commands

- docker compose up – Start services.
- docker compose down – Stop and remove services.
- docker compose ps – List running services.
- docker compose logs – View logs.
- docker compose build – Build images.

Compose File

Docker Compose uses a **YAML (docker-compose.yml)** file to define services, networks, and volumes.

Docker Engine

What is Docker Engine?

Docker Engine is the core component of Docker responsible for building, running, and managing containers.

Components

- **Docker CLI:** Command-line interface used to interact with Docker.
- **Docker Daemon:** Background service that manages containers and images.
- **REST API:** Allows applications to communicate with the Docker daemon.

Docker Storage

Docker provides storage options for preserving data.

Storage Drivers

Storage drivers manage how Docker images and containers store data.

Data Volumes

Volumes store persistent data independently of containers.

Common Volume Commands

- docker volume create – Create a volume.
- docker volume ls – List all volumes.

Changing Storage Driver

Docker storage drivers can be changed through Docker Engine configuration based on system requirements.

Docker Networking

Docker networking enables communication between containers and external systems.

Default Docker Networks

- Bridge Network
- Host Network
- None Network

Network Commands

- docker network ls – List all networks.
- docker network inspect <network> – View network details.
- docker network create mynetwork – Create a new network.

Container Orchestration

What is Container Orchestration?

Container orchestration is the automated management of containerized applications, including deployment, scaling, networking, and monitoring.

Why is it Needed?

- Automates container deployment.
- Manages large numbers of containers.
- Improves availability.
- Simplifies scaling.
- Supports fault recovery.

Benefits

- Automated deployment.
- Load balancing.
- High availability.
- Automatic scaling.
- Self-healing containers.
- Efficient resource utilization.

Kubernetes

Kubernetes is the most popular container orchestration platform. It automates deployment, scaling, networking, and management of Docker containers across multiple servers.

Container Orchestration vs Docker

Docker	**Container Orchestration**
Creates and runs containers.	Manages multiple containers.
Best for single-container applications.	Best for large-scale applications.
Provides containerization.	Provides automation, scaling, and management.
Example: Docker Engine	Example: Kubernetes

Summary

Docker is an open-source containerization platform that packages applications and their dependencies into lightweight containers for consistent deployment. It provides powerful commands to manage containers and images, supports Docker Compose for multi-container applications, Docker Engine for container management, storage volumes for persistent data, and networking for container communication. For large-scale deployments, container orchestration tools such as **Kubernetes** automate container deployment, scaling, monitoring, and management, making Docker an essential technology in modern DevOps and cloud-native application development.